

Artificial Intelligence

Internship — Task Submission

Software Development Company

Intern Name	Jay Patel
Internship Domain	Artificial Intelligence
Organisation	CodeAlpha
Tasks Completed	Task 1, Task 2, Task 4 (3 of 4)
GitHub Repo	github.com/jaypatel29042008-glitch/CodeAlpha_AI_Internship
LinkedIn Profile	linkedin.com/in/jay-patel-01ab3b377
Submission Date	June 2025

Tasks Overview

■ Task 1	Language Translation Tool	Google Translate API + Tkinter UI
■ Task 2	Chatbot for FAQs	TF-IDF + Cosine Similarity (NLP)
■ Task 4	Object Detection & Tracking	YOLOv8 + Centroid Tracker

Objective

Build a desktop application where users can type text, select source and target languages, and receive real-time translations using the Google Translate API. The interface provides a copy button, auto-detection of source language, and displays detected language.

Technologies Used

Library / Tool	Version	Purpose
googletrans	4.0.0-rc1	Google Translate API wrapper
tkinter	stdlib	Desktop GUI framework
Python	3.10+	Core programming language

Architecture & Workflow

1. User enters text in the input field and selects source/target languages from dropdown menus.
2. On clicking 'Translate', the text is passed to the **googletrans.Translator** instance.
3. The library sends an HTTPS request to Google Translate and returns a **Translated** object.
4. The translated text is displayed in the output box; detected language is shown below.
5. The 'Copy' button appends the result to the clipboard using **root.clipboard_append()**.

Key Code Snippet — Translation Logic

```
from googletrans import Translator, LANGUAGES
translator = Translator()

def translate_text():
    src = LANG_OPTIONS.get(src_combo.get(), "auto")
    dest = LANG_OPTIONS.get(dst_combo.get(), "en")
    result = translator.translate(input_text, src=src, dest=dest)
    output_box.insert(tk.END, result.text)
    detected_label.config(text=LANGUAGES[result.src].title())
```

Features Implemented

- Support for 100+ languages via Google Translate API
- Auto-detect source language
- Copy to clipboard button
- Clear all fields button
- Dark-themed, modern Tkinter UI
- Detected language label (shows the auto-detected source language)

File

- **task1_translation.py** — self-contained, run with Python 3.10+

Objective

Develop an NLP-powered chatbot that matches user questions to the most relevant FAQ answer using TF-IDF vectorization and cosine similarity. The chatbot covers 20 FAQs on AI, ML, NLP, Python, and the CodeAlpha internship program.

Technologies Used

Library / Tool	Purpose
scikit-learn	TfidfVectorizer + cosine_similarity
numpy	Array operations for similarity scores
tkinter	Chat UI with ScrolledText widget
Python 3.10+	Core language

NLP Pipeline

1. **Knowledge Base:** 20 curated FAQ pairs (question → answer) stored in a Python dict.
2. **Vectorisation:** Questions are transformed into TF-IDF vectors at startup.
3. **Query matching:** User input is vectorised and compared to all FAQ vectors via cosine similarity.
4. **Threshold:** If the best match score < 0.15, a fallback 'I don't know' response is shown.
5. **Greeting/Exit detection:** Common phrases (hi, bye) are handled before the NLP pipeline.
6. **Confidence display:** Each answer shows the match confidence percentage.

Key Code Snippet — Cosine Similarity Matching

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

vectorizer = TfidfVectorizer(stop_words="english")
tfidf_matrix = vectorizer.fit_transform(questions) # build FAQ index

def get_response(user_input):
    query_vec = vectorizer.transform([user_input])
    scores = cosine_similarity(query_vec, tfidf_matrix).flatten()
    best_idx = int(np.argmax(scores))
    if scores[best_idx] < 0.15:
        return "I'm not sure. Please rephrase your question."
    return answers[best_idx]
```

Sample FAQ Topics Covered

AI / ML Concepts	Deep Learning, Neural Networks, Overfitting, Underfitting
NLP & CV	Natural Language Processing, Computer Vision, LLMs
Python & Tools	Python libraries (TensorFlow, PyTorch, OpenCV, NLTK)

Internship Info	Submission process, task requirements, CodeAlpha overview
Object Detection	YOLO algorithm, OpenCV capabilities

File

■ **task2_chatbot.py** — self-contained, run with Python 3.10+

TASK 4

Object Detection and Tracking

Objective

Build a real-time object detection and tracking pipeline using YOLOv8 (nano model) for frame-level detection and a custom centroid-based tracker for consistent object IDs across frames. Bounding boxes, labels, confidence scores, tracking IDs, and motion trails are all rendered on the live video feed.

Technologies Used

Library / Tool	Version	Purpose
ultralytics	8.x	YOLOv8 model — detection
OpenCV	4.x	Video capture, frame drawing
numpy	1.x	Distance matrix computation
Python	3.10+	Core language

System Architecture

- Input:** Webcam stream (or video file) captured frame-by-frame via `cv2.VideoCapture`.
- Detection:** Each frame is passed to **YOLOv8n** (80-class COCO model); returns bounding boxes, class labels and confidence scores.
- Tracking:** Bounding box centroids are fed to the **CentroidTracker**, which assigns persistent IDs using Euclidean-distance matching.
- Trail rendering:** A deque of past centroid positions per ID is drawn as a fading polyline — making movement paths visible.
- HUD overlay:** Frame index, active object count, and total unique objects tracked are shown at the top of the frame.
- Output:** Annotated frames displayed in real-time; 's' key saves screenshot.

Key Code Snippet — Detection + Tracking Loop

```
model = YOLO("yolov8n.pt") # load pretrained model
tracker = CentroidTracker(max_disappeared=25)

while True:
    ret, frame = cap.read()
    results = model(frame, conf=0.45, iou=0.45)[0] # detect
    bboxes = [map(int, box.xyxy[0]) for box in results.bboxes]
    tracked = tracker.update(bboxes) # assign IDs

    for oid, centroid in tracked.items():
        cv2.rectangle(frame, (x1,y1), (x2,y2), id_color(oid), 2)
        cv2.putText(frame, f"ID:{oid} {label}", (x1,y1-5), ...)
        draw_trail(frame, tracker.trails[oid], id_color(oid))
```

Centroid Tracker — Algorithm

The **CentroidTracker** class maintains a dictionary of active object IDs and their centroid coordinates. On each frame it: (1) computes an N×M Euclidean distance matrix between existing and new centroids, (2) greedily matches them row-by-row by minimum distance, (3) registers unmatched new centroids as new tracks, and (4) increments a 'disappeared' counter for unmatched existing tracks, removing them after 25 frames of absence. Motion trails are stored per-ID in fixed-length deques.

Configuration Options

Parameter	Default	Description
VIDEO_SOURCE	0	0=webcam; string path for video file
CONF_THRESH	0.45	YOLOv8 confidence threshold
IOU_THRESH	0.45	YOLOv8 NMS IoU threshold
MAX_TRAIL_LEN	30	Number of past frames shown in trail
MODEL_SIZE	yolov8n	n=nano; s/m/l for larger models

File

■ **task4_object_detection.py** — run with Python 3.10+; requires CUDA GPU for best performance

Summary & GitHub Structure

All three tasks have been implemented as fully functional, well-commented Python applications. Each project demonstrates practical AI/ML skills and meets the CodeAlpha internship completion criteria (minimum 2 of 4 tasks).

Recommended GitHub Repository Structure

```
jaypatel29042008-glitch/CodeAlpha_AI_Internship/
```

```
■■■ README.md
```

```
■■■ requirements.txt
```

```
■■■ Task1_LanguageTranslation/
```

```
■ ■■■ task1_translation.py
```

```
■ ■■■ README.md
```

```
■■■ Task2_FAQChatbot/
```

```
■ ■■■ task2_chatbot.py
```

```
■ ■■■ README.md
```

```
■■■ Task4_ObjectDetection/
```

```
■■■ task4_object_detection.py
```

```
■■■ README.md
```

requirements.txt

```
googletrans==4.0.0-rc1
```

```
scikit-learn>=1.3
```

```
numpy>=1.24
```

```
opencv-python>=4.8
```

```
ultralytics>=8.0
```

Task	Title	Status	Filename
1	Language Translation Tool	■ Complete	task1_translation.py
2	Chatbot for FAQs	■ Complete	task2_chatbot.py
3	Music Generation with AI	■ Skipped	—
4	Object Detection & Tracking	■ Complete	task4_object_detection.py